

```

sc = SparkContext(appName='WordsInFiles')
data = sc.wholeTextFiles('hdfs://h_mstr/user/fedora/docs')
word_index = data.flatMap(wholeFile).reduceByKey(lambda
a,b:','.join((a,b)))
word_index.persist()
output=word_index.collect()
output.sort()
# print the first 10 values
for i in range(10):
    print output[i]
explore(word_index)
sc.stop()

```

The function *wholeFile* returns a list of (word, file name) pairs for unique words. As earlier, the text has been converted to lower case and all special characters replaced by a space.

The *reduceByKey* function joins all the file names with a comma as the separator. For verification, the first 10 results of the sorted output are printed.

The *persist* method saves the result of *word_index*, which is useful if you want to use the same data set again. For example, in the *explore* method mentioned above, you may want to analyse a list of words and see how many of these words appear in a file.

```

class filter_keys():
    def __init__(self, keys):
        self.keys = keys
    def filter(self, x):
        return x[0] in self.keys
def explore_keys(rdd, keys):
    example = filter_keys(keys)
    res = rdd.filter(example.filter) \
        .flatMap(lambda x:x[1].split(',')) \
        .map(lambda x:(x,1)) \
        .reduceByKey(lambda a,b:a+b)
    out=res.collect()
    out.sort(lambda a,b:-a[1]_cmp_(b[1]))
    print keys,len(out)
    print out
def explore(rdd):
    explore_keys(rdd,['linux','fedora','ubuntu'])
    explore_keys(rdd,['linux','fedora','ubuntu','python'])

```

The first step is to select only those records that pertain to the keywords. The *Filter* operation on the data set will do just that. However, we need to pass a list of words as a parameter and this can be achieved by creating the class *filter_keys*. The class is initialised with a list of keywords

and the filter method in the class can use this list.

The next step is like the word count example. Each line is a word followed by a string of comma-separated file names. So, split the file names into a list and count the number of times each file is selected.

Sort the results in descending order (of the frequency) and print them. You can repeat the exercise for various keywords and the intermediate data set will be reused without the need to save it in a file. This is one of the major advantages of Spark over MapReduce.

From a developer's perspective, the ability to code the entire analysis (which may consist of a sequence of MapReduce operations) in a single file makes it much easier to think through the problem and write better code. 

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com> and <http://sethanil.blogspot.com>, or reach him via email at anil@sethanil.com.

EMBEDDED SOFTWARE DEVELOPMENT COURSES AND WORKSHOPS

School of Embedded Software
www.schoolofembedded.org

COURSES:

- Embedded System Programming on Arm-Cortex-M
- ARM Cortex-M architecture, internals and Programming
- Embedded OS internals and programming
- BSP and drivers
- Linux Internals for Driver Development - in April 2015
- Linux Device Drivers

VENUE:

School of Embedded Software Development
Kalyan Nagar, HRBR Layout, Bangalore - 560043

Faculty : **Babu Krishnamurthy**

Email : babu_krishnamurthy@schoolofembedded.org

Phone : +91-98453-65845/080-41207855

COURSE DETAILS:

http://www.schoolofembedded.org/course_s

<http://www.schoolofembedded.org/events-default>